hs-bindgen

Automatic Generation of C Bindings for Haskell

Joris Dral

October 26, 2026 — Copyright © 2026 Well-Typed LLP

Well-Typed
The Haskell Consultants

Foreign Function Interface (FFI)

One of Haskell's strengths is that it can interface with foreign code (primarily C).

This is enables us to use fast, battle-tested, industrial-strength C libraries in Haskell projects.

The interfacing is not free: it requires us to write bindings.

Writing bindings by hand is tedious: large parts are mechanical, and bindings have to be kept up to date with new releases of C libraries.

Writing bindings by hand is error-prone: run-time mismatches may occur.

Introducing `hs-bindgen`

`hs-bindgen` ...

- automatically generates Haskell bindings from C header files.
- leverages the power of `libclang`.
- is structured like a compiler.
- extracts and translates documentation.
- supports translation of multiple C libraries in a compositional way.
- supports translation of macros to values, functions, and types.
- has a manual and examples.
- is a team effort, with many community contributions.
- is on Hackage: version 1.0.0.0 was released recently.

More features

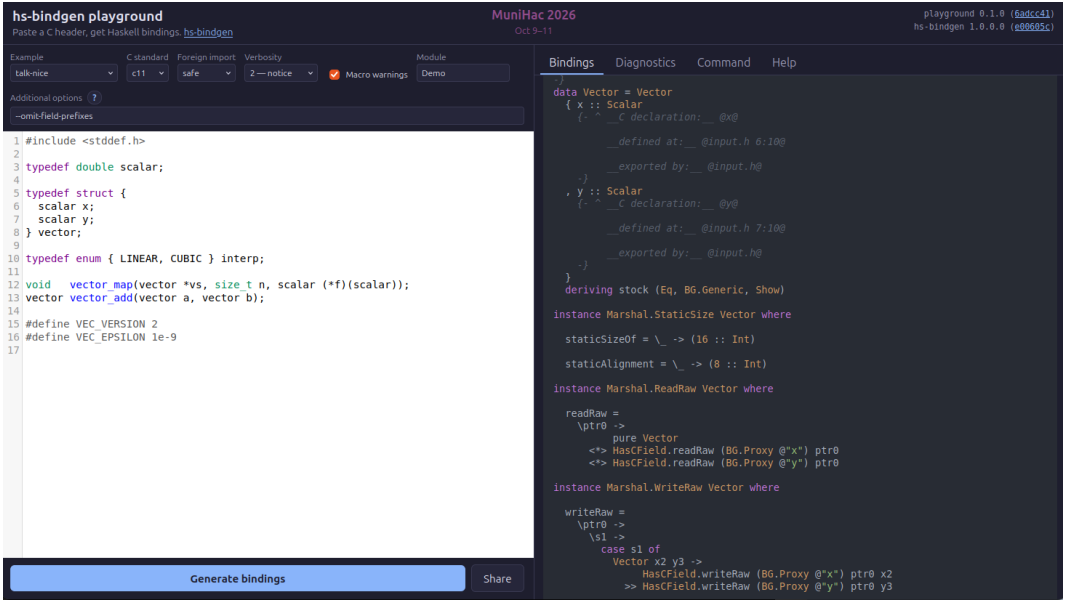
`hs-bindgen` supports...

- most of the C standard, including...
  - structs, unions, enums, global variables, functions.
  - functions that pass structs or unions by value.
  - anonymous structs and unions.
  - bit-fields.
  - arrays.
- customizability using prescriptive binding specifications.
- compositionality using external binding specifications.
- output modes: pretty-printing, template Haskell.
- selecting declarations based on predicates.
- program slicing.
- record-dot syntax.
- pointer manipulation (with record-dot syntax).
- squashing of declarations.
- a predictable naming algorithm for Haskell declarations.
- detecting conflicting Haskell names.



Compiling C to Haskell – How hs-bindgen Translates C Headers, by Dominik Schrempf

The playground



https://bindgen.well-typed.com/

Real-world example: Botan

Botan is a C++ cryptography library with a C API.

C API in `ffi.h`: **2669** lines of C code.

C++ API in all public headers: **42891** lines of C++ code.

C++ implementation: many more lines of code...

`botan-bindings` on Hackage: **4827** lines of Haskell code.

Bindings generated by `hs-bindgen`: **47226** lines of Haskell code.

Botan RNG

Botan has an API for random number generation. Let's use it as a case study!

```
1 typedef struct botan_rng_struct* botan_rng_t;
2 int botan_rng_init(botan_rng_t* rng, const char* rng_type);
3 int botan_rng_get(botan_rng_t rng, uint8_t* out, size_t out_len);
4 int botan_rng_destroy(botan_rng_t rng);
```

Things to note:

- `struct botan_rng_struct` is opaque.
- All functions use error return codes.
- Strings are used to set options such as `rng_type`.

Invocation

To generate bindings (for RNG-related functions only):

```
FLAGS=(
  -I "$BOTAN_INCLUDE_DIR"
  --clang-option '-std=gnu17'
  --hs-output-dir "$HS_DIR"
  --module "Generated.Botan.RNG"
  --select-by-decl-name "botan_rng_*"
  --enable-program-slicing
  # ...
)

hs-bindgen-cli preprocess "${FLAGS[@]}" 'botan/ffi.h'
```

Output

The simplified output:

```
data Botan_rng_struct
newtype Botan_rng_t = Botan_rng_t (Ptr Botan_rng_struct)
  deriving (Storable, ...)
botan_rng_init    :: Ptr Botan_rng_t -> PtrConst CChar -> IO CInt
botan_rng_get     :: Botan_rng_t     -> Ptr Word8 -> CSize -> IO CInt
botan_rng_destroy :: Botan_rng_t     -> IO CInt
```

Things to note:

- The translation is fully automatic.
- Compositionality in action: we use `CSize` for `size_t` rather than generating a new type.
- `hs-bindgen` does not treat `size_t` specially. It has a built-in external binding specification.

Low-level and high-level bindings

`hs-bindgen` currently generates **low-level** bindings that are faithful to the original C API.

In Haskell we like to use **high-level** bindings…

- without explicit memory management.
- without pointer manipulation.
- with exceptions or `Either`/`Maybe` instead of error codes.

So we're not there yet!

A second look

Let's take a second look at `botan_rng_init`

```
1 int botan_rng_init(botan_rng_t* rng, const char* rng_type);
```

It is not clear from the code, but we (the developers) know that...

- the return `int` is an error code.
- `rng` is a buffer that is used to report the resulting RNG context back.

What is **objectively** clear from the code is that...

- `rng_type` is a pointer to `const`, so it can't be an output parameter.

By hand

```
newtype Botan_rng_t = Botan_rng_t (Ptr Botan_rng_struct)
  deriving (Storable, ...)
botan_rng_init    :: Ptr Botan_rng_t -> PtrConst CChar -> IO CInt
```

Using our domain-specific knowledge, we can write:

```
newtype RNG = RNG Botan_rng_t
rngInit :: String -> IO RNG
rngInit rngType =
  alloca $ \rngPtr ->
    withCString rngType $ \rngTypePtr -> do
      throwErrnoIfNegative "botan_rng_init" $
        botan_rng_init rngPtr (PtrConst.unsafeFromPtr rngTypePtr)
      RNG <$> peek rngPtr
throwErrnoIfNegative :: String -> IO CInt -> IO ()
```

Getting out of hand

Writing high-level bindings from scratch quickly gets out of hand.

We have to...

- allocate input and output buffers.
- write to input buffers.
- call the foreign function using the buffers.
- check status codes, and possibly throw exceptions.
- read from output buffers.
- free buffers.

And this is all fairly repetitive to write. Surely this should be easier?

A bad case

```haskell
srp6GenerateVerifier ::
     String
  -> String
  -> [Word8]
  -> String
  -> String
  -> CSize
  -> IO [Word8]
srp6GenerateVerifier username password salt groupId hashId maxLen =
    withCString username $ \usernamePtr ->
      withCString password $ \passwordPtr ->
        withArray salt $ \saltPtr ->
          let saltLen = fromIntegral (length salt) in
          withCString groupId $ \groupIdPtr ->
            withCString hashId $ \hashIdPtr ->
              allocaBytes (fromIntegral maxLen) $ \verifierPtr ->
                with maxLen $ \verifierLenPtr -> do
                  throwErrnoIfNegative "botan_srp6_generate_verifier" $
                    botan_srp6_generate_verifier
                      (PtrConst.unsafeFromPtr usernamePtr)
                      (PtrConst.unsafeFromPtr passwordPtr)
                      (PtrConst.unsafeFromPtr saltPtr)
                      saltLen
                      (PtrConst.unsafeFromPtr groupIdPtr)
                      (PtrConst.unsafeFromPtr hashIdPtr)
                      verifierPtr
                      verifierLenPtr
                  verifierLen <- peek verifierLenPtr
                  peekArray (fromIntegral verifierLen) verifierPtr
```

Introducing `binding-combinators`

`binding-combinators`...

- provides combinators for writing high-level bindings declaratively.
- is composable: combinators can be composed from smaller combinators.
- is extensible: new combinators can be added.
- is readable: combinators can be read from top to bottom.
- enables full expressivity of Haskell: combinators are first-class Haskell values that are type-checked.
- guides the user in writing bindings using the Haskell type system (and HLS).
- works with any FFI bindings, not just ones generated by `hs-bindgen`.
- is not yet on Hackage.
- https://github.com/well-typed/binding-combinators

Using combinators (1)

```
newtype Botan_rng_t = Botan_rng_t (Ptr Botan_rng_struct)
  deriving (Storable, ...)
botan_rng_init    :: Ptr Botan_rng_t -> PtrConst CChar -> IO CInt
```

Using `binding-combinators`, we can write:

```
rngInit_V2 :: String -> IO RNG
rngInit_V2 =
      toHighLevel botan_rng_init
    $ output        (unmarshalOutPure RNG)
    $ input         withCStringIn
    $ checkedResult (\ret ->
        throwErrnoIfNegative "rngInit_v2" (pure ret))
```

Using combinators (2)

```haskell
rngInit_V2 :: String -> IO RNG
rngInit_V2 =
      toHighLevel botan_rng_init
    $ output        (unmarshalOutPure RNG)
    $ input         withCStringIn
    $ checkedResult (\ret ->
        throwErrnoIfNegative "rngInit_v2" (pure ret))
```

Things to note:

- `toHighLevel` converts a low-level binding to a high-level binding using a **specification**.
- `output` declares an output parameter using an **unmarshaller**.
- `input` declares an input parameter using an **marshaller**.
- `checkedResult` is a **closer** that acts on the function return value.

A better bad case

```
srp6GenerateVerifier_V2 ::
     String
  -> String
  -> [Word8]
  -> String
  -> String
  -> CSize
  -> IO [Word8]
srp6GenerateVerifier_V2 =
    toHighLevel botan_srp6_generate_verifier
    $ input   withCStringIn
    $ input   withCStringIn
    $ input2  withArrayLenIn
    $ input   withCStringIn
    $ input   withCStringIn
    $ {- 2 -} withArrayLenInOut
    $ checkedResult (\ret ->
        throwErrnoIfNegative "srp6GenerateVerifier_V2" (pure ret))
```

Next steps for high-level bindings

In the future, we plan to generate high-level bindings using heuristics.

We are thinking about making these heuristics customizable using hooks.

API description languages could feed into these hooks as well.

```
/*!
 * Copies chars from a source string to a destination string.
 * \param[out] dest The string to copy to.
 * \param[in]  src  The string to copy from.
 * \param[in]  n    The number of bytes to copy
 */
void strcpy(char *dest, const char *src, size_t n);
```

Future work

We are also planning to support...

- project files.
- generating Cabal files.
- build process integration.
- `hs-bindgen` as a library, which unblocks...
  - customizable macro languages.
- ...

Related work

- `rust-bindgen`: Generate Rust bindings from C header files.
- `hsc2hs`: Integrate small C snippets into Haskell code.
- `c2hsc`: Generate `.hsc` files from C header files.
- `c2hs`: An interfacing tool that eases Haskell access to C libraries. Analyses C header files to fill in missing details in a template file.